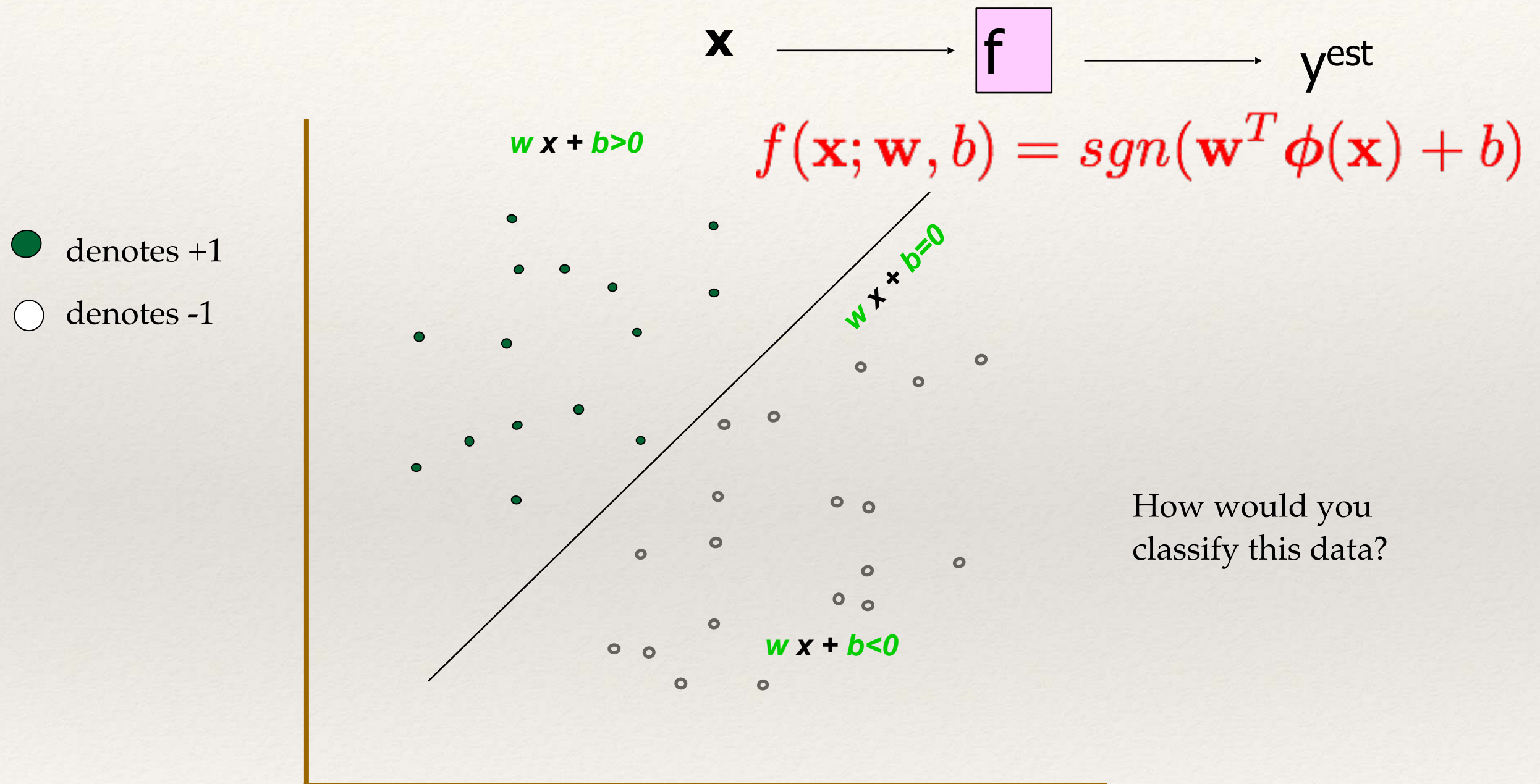


E9 205 Machine Learning for Signal Processing

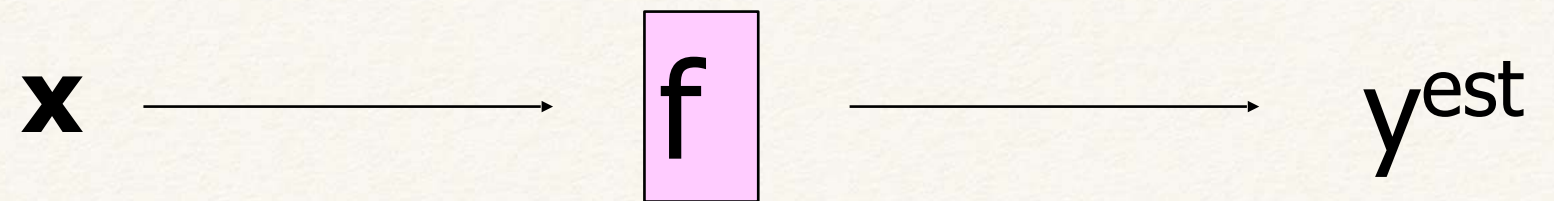
Support Vector Machines

19-10-2016

Linear Classifiers

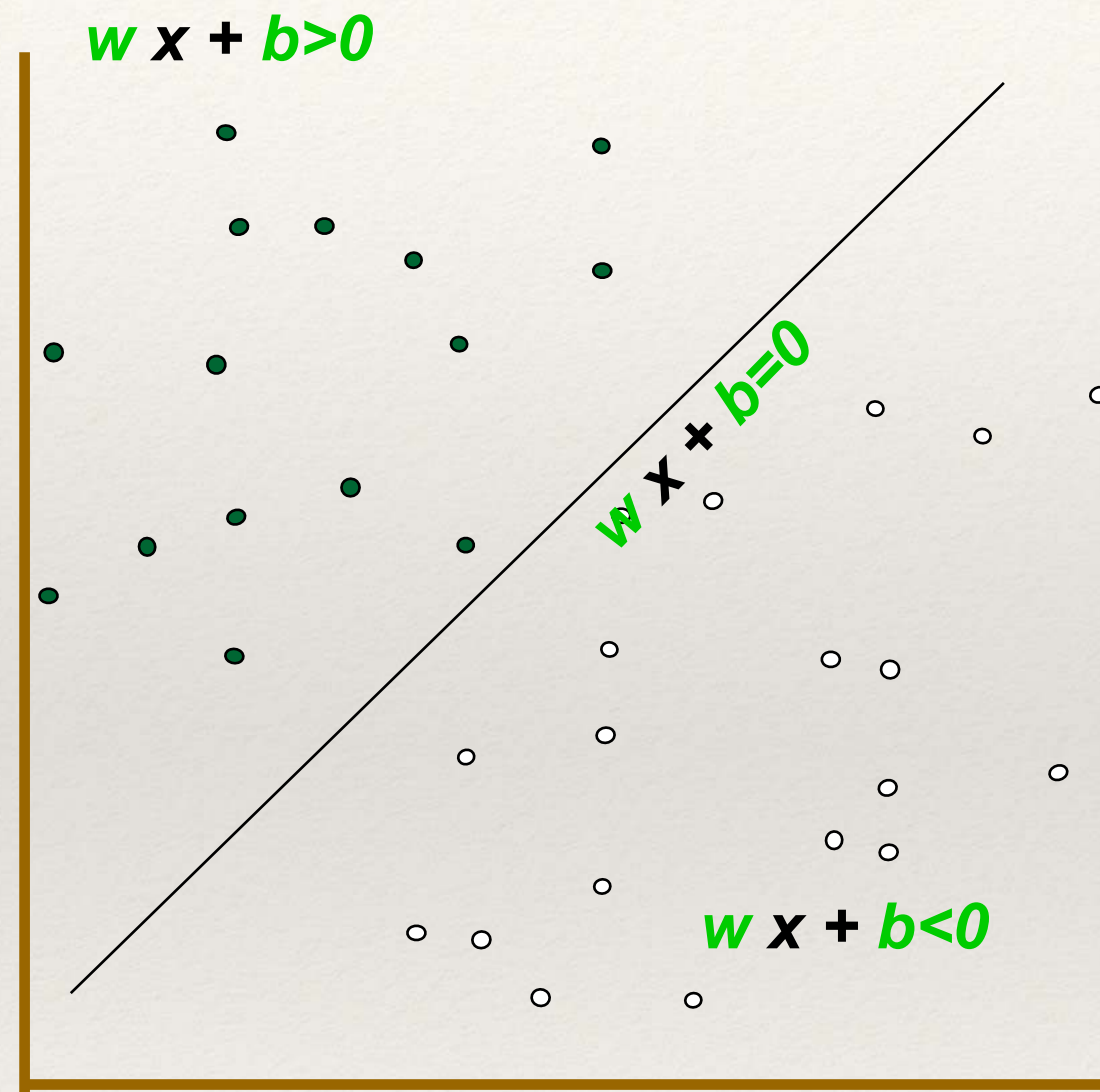


Linear Classifiers



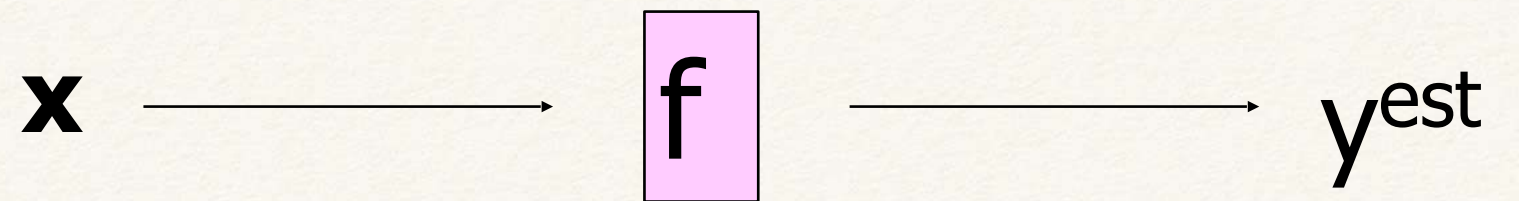
- denotes +1
- denotes -1

$$f(\mathbf{x}; \mathbf{w}, b) = \text{sgn}(\mathbf{w}^T \phi(\mathbf{x}) + b)$$



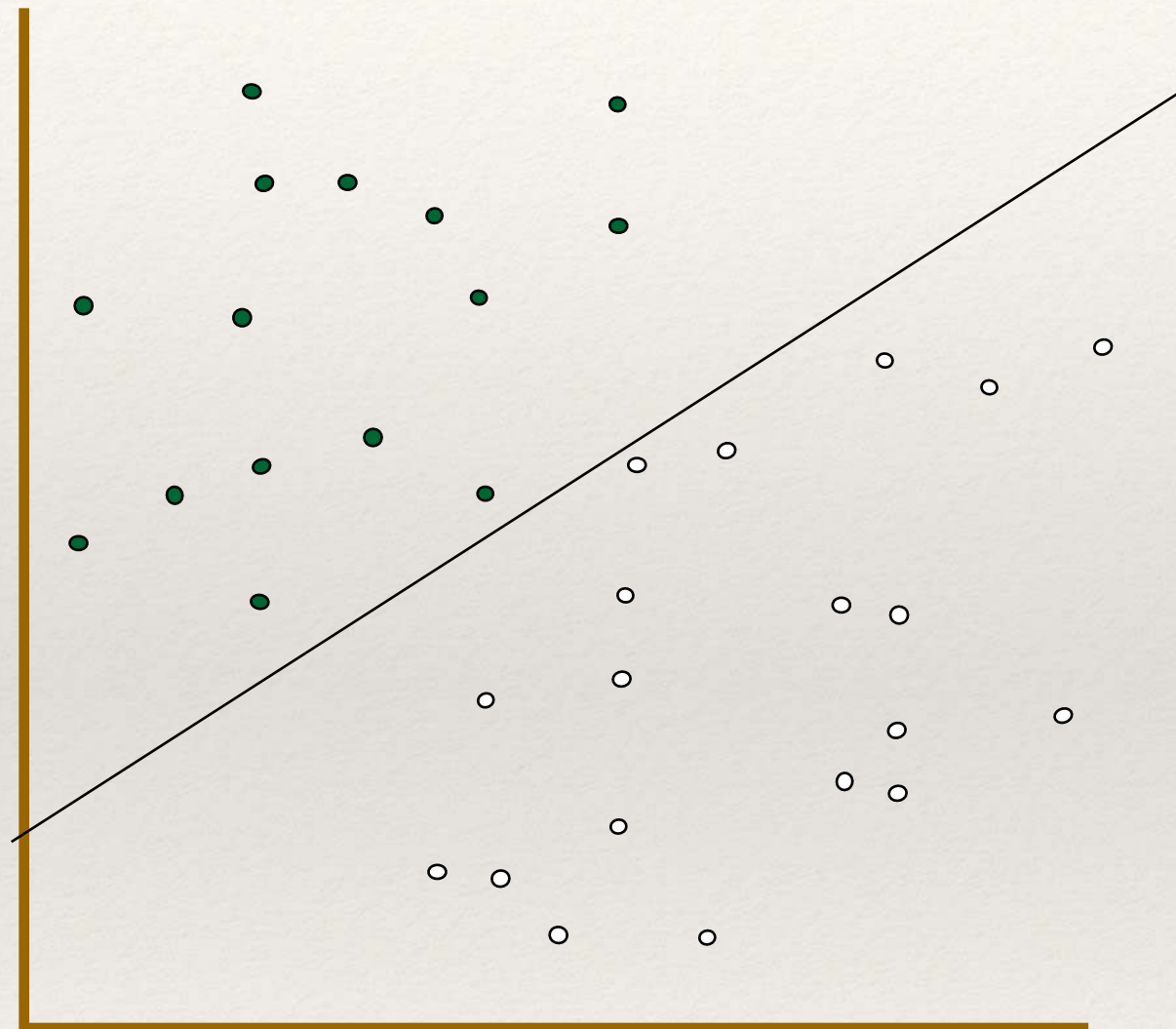
How would you classify this data?

Linear Classifiers



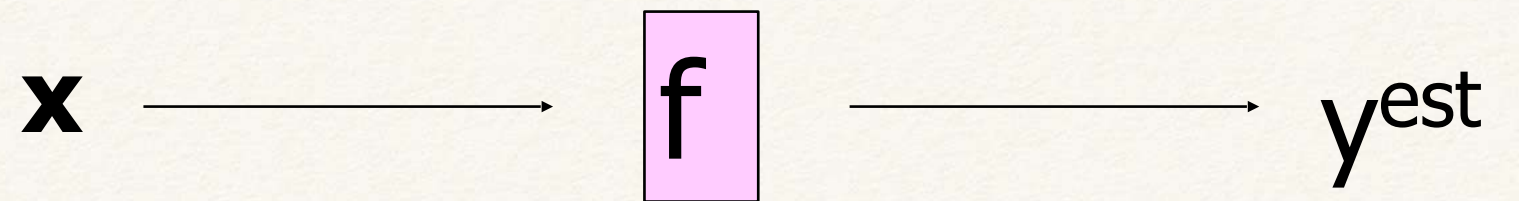
$$f(\mathbf{x}; \mathbf{w}, b) = \text{sgn}(\mathbf{w}^T \phi(\mathbf{x}) + b)$$

- denotes +1
- denotes -1

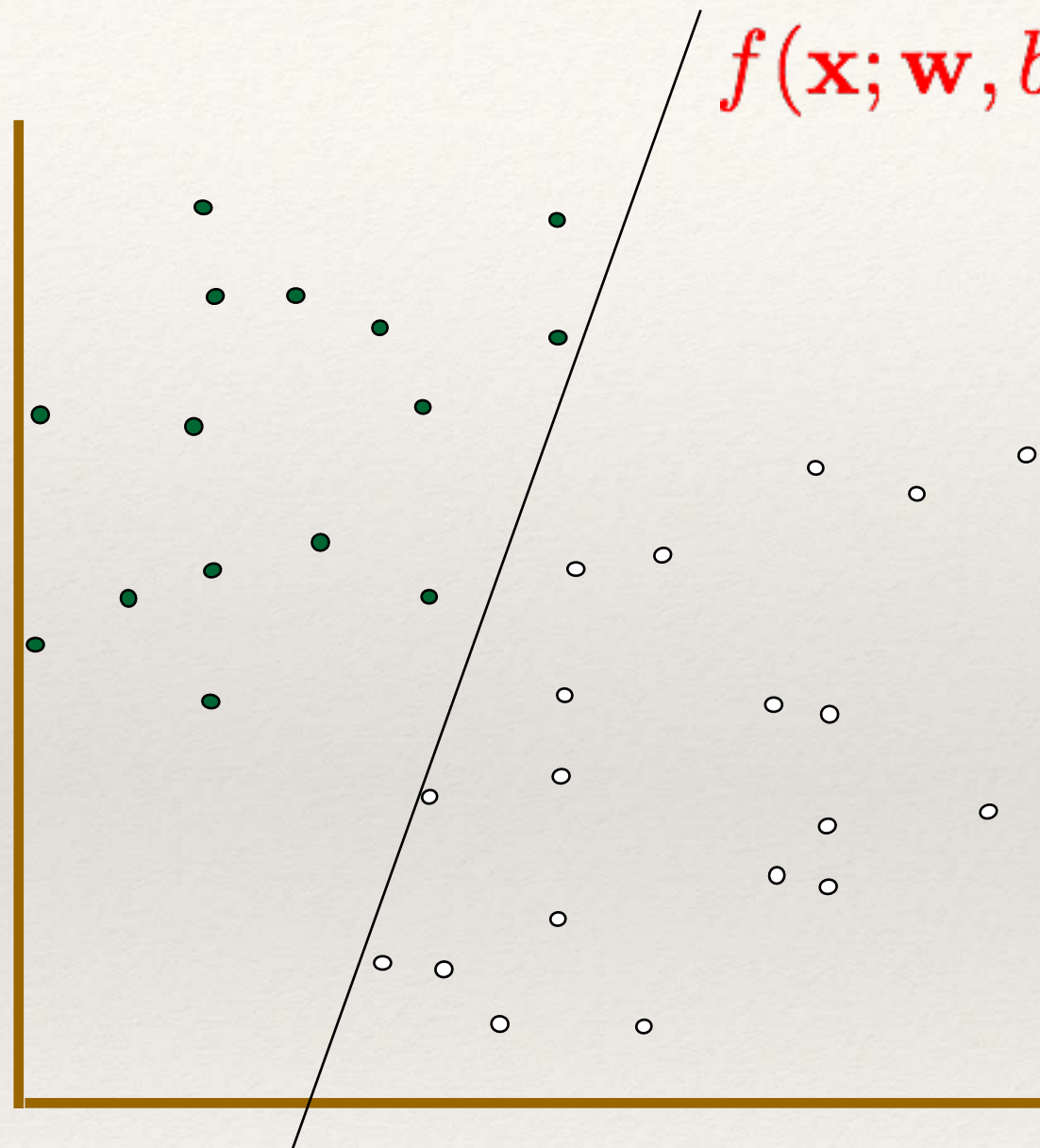


How would you
classify this data?

Linear Classifiers



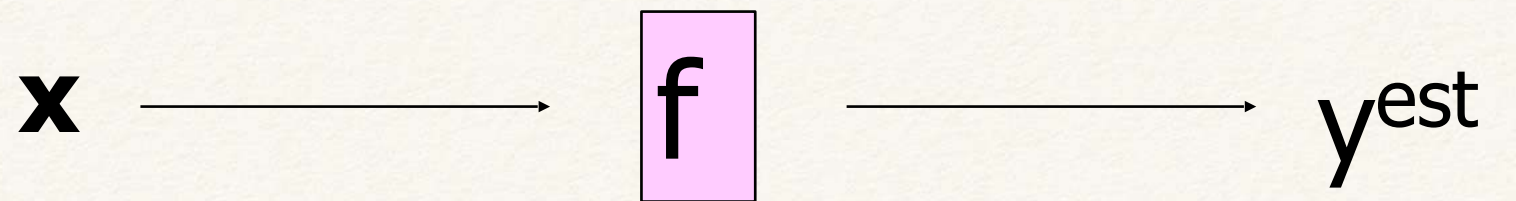
- denotes +1
- denotes -1



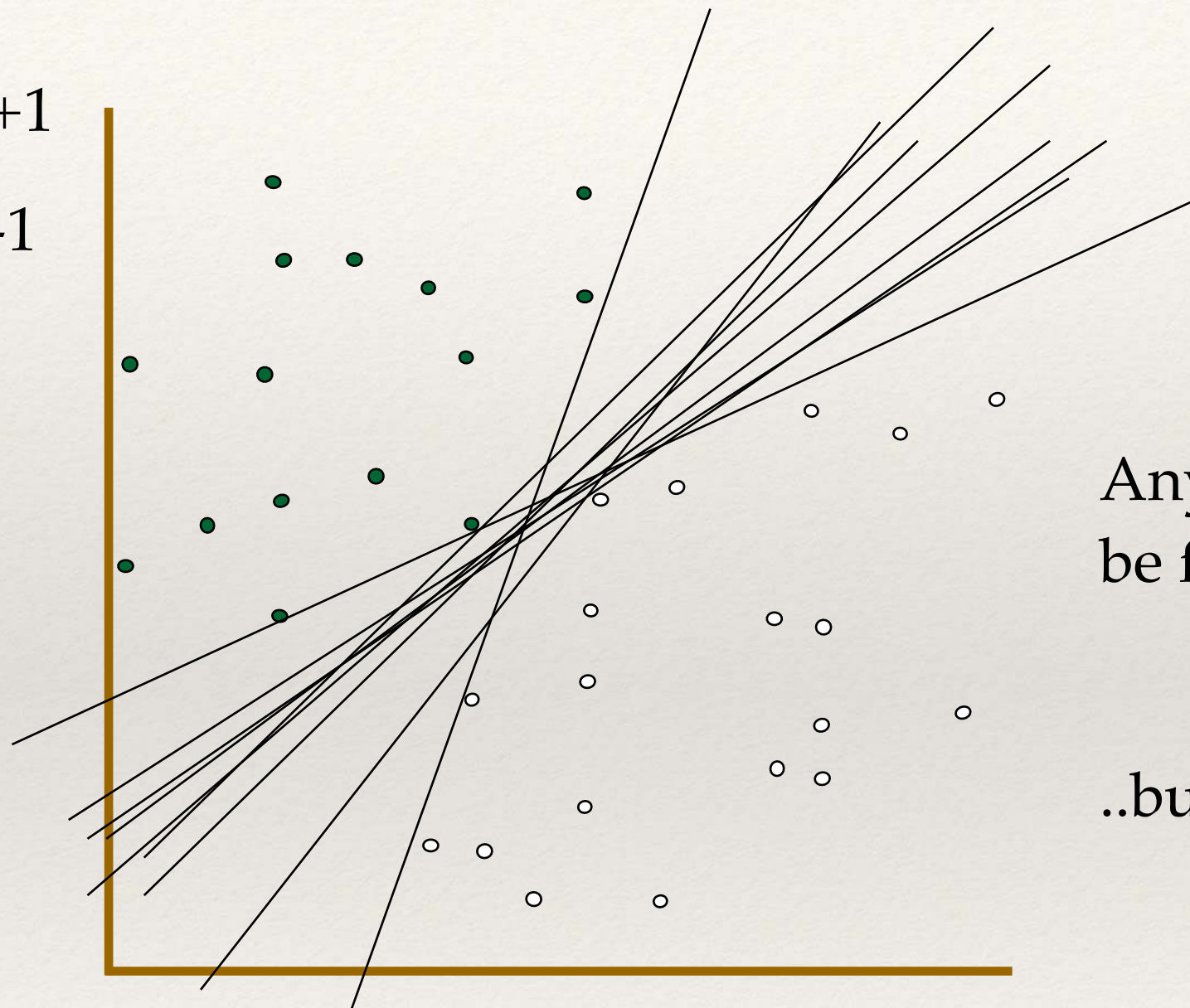
$$f(\mathbf{x}; \mathbf{w}, b) = \text{sgn}(\mathbf{w}^T \phi(\mathbf{x}) + b)$$

How would you
classify this data?

Linear Classifiers



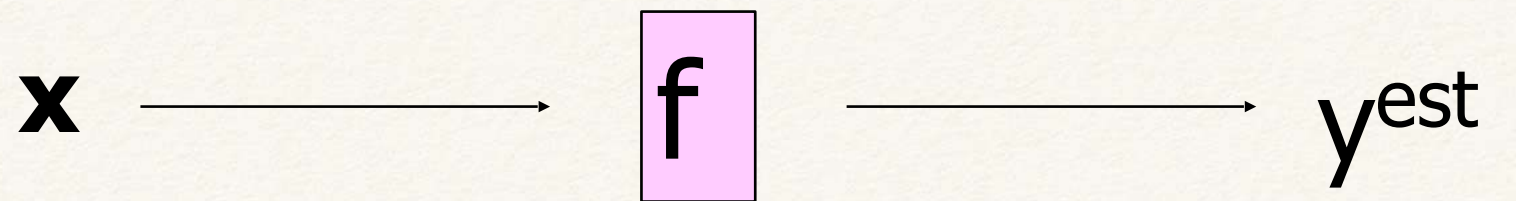
- denotes +1
- denotes -1



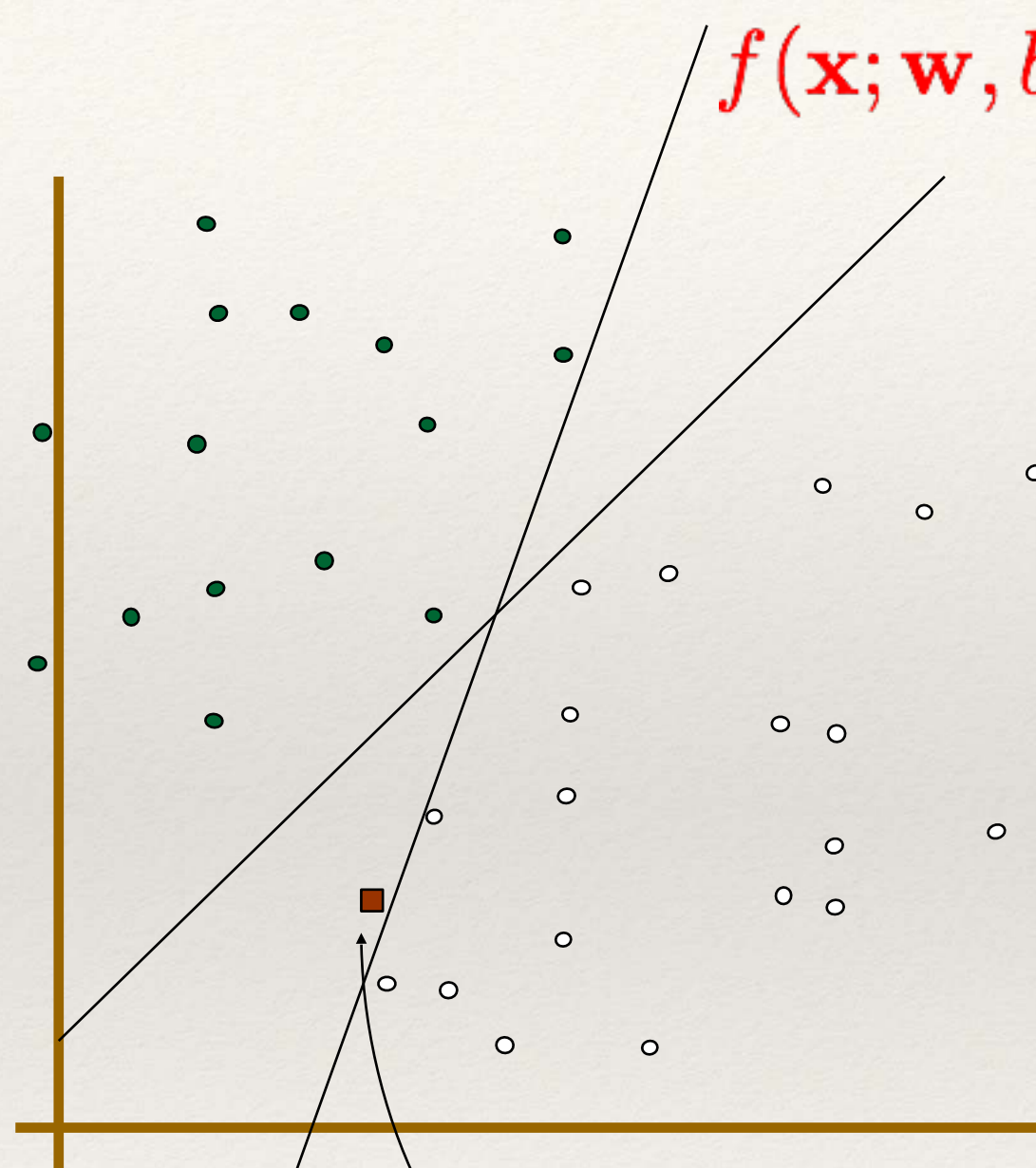
Any of these would be fine..

..but which is best?

Linear Classifiers



- denotes +1
- denotes -1

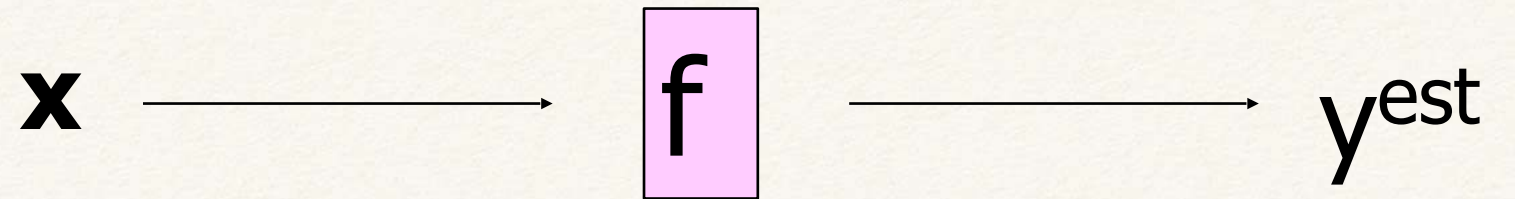


$$f(\mathbf{x}; \mathbf{w}, b) = \text{sgn}(\mathbf{w}^T \phi(\mathbf{x}) + b)$$

How would you classify this data?

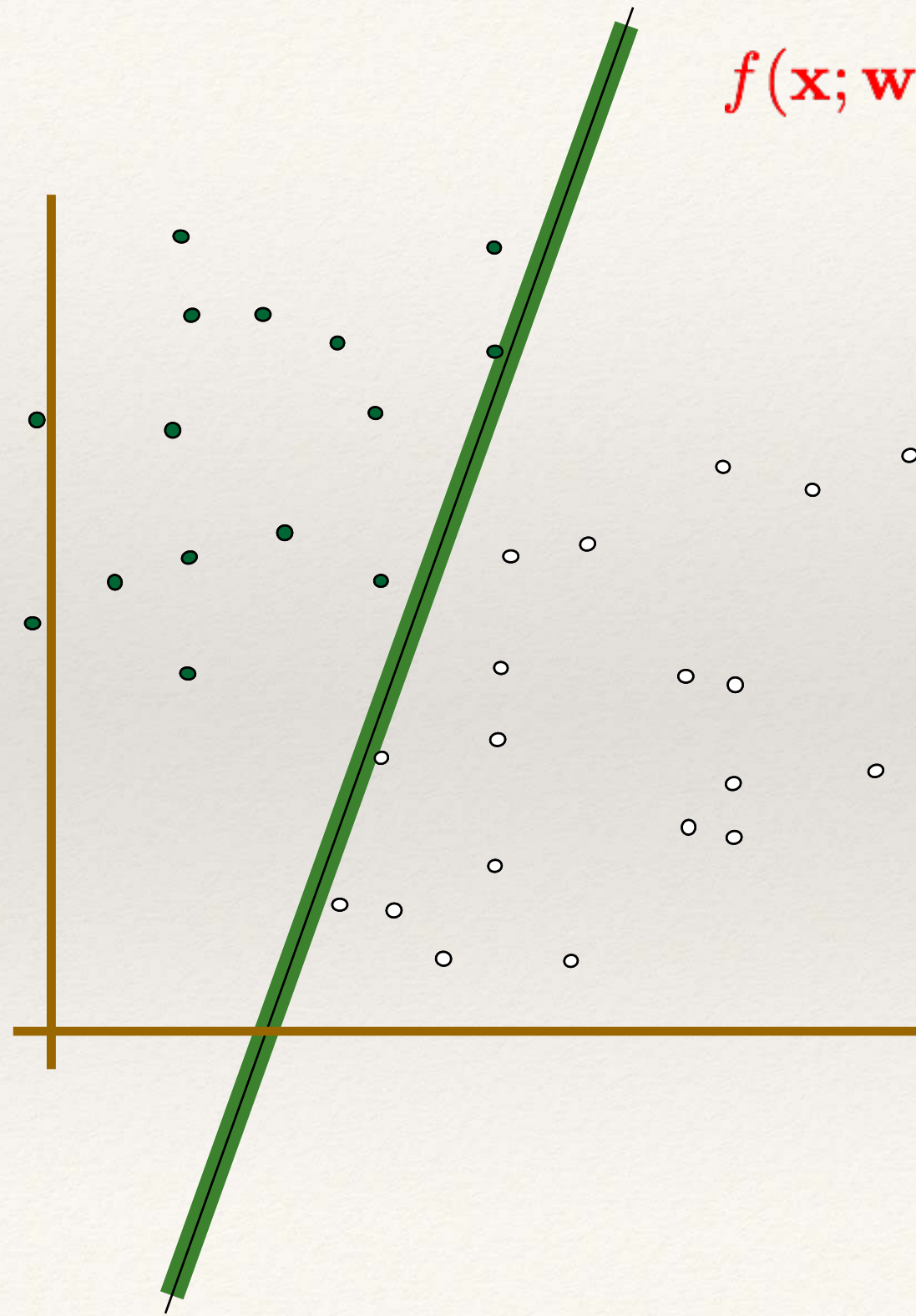
Misclassified
to +1 class

Linear Classifiers



$$f(\mathbf{x}; \mathbf{w}, b) = \text{sgn}(\mathbf{w}^T \phi(\mathbf{x}) + b)$$

- denotes +1
- denotes -1



Define the **margin** of a linear classifier as the width that the boundary could be increased by before hitting a datapoint.

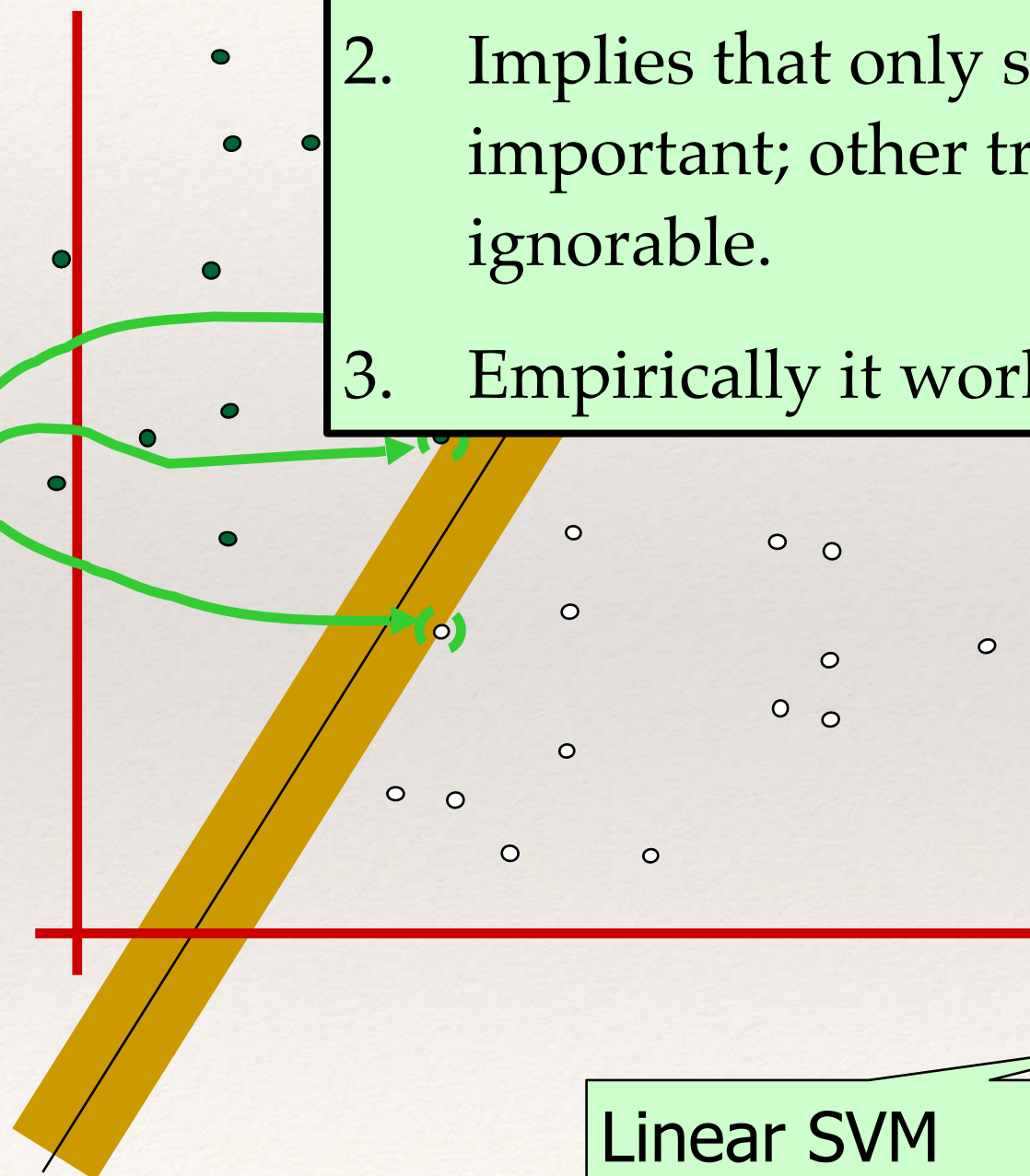
Maximum Margin



- denotes +1
- denotes -1

1. Maximizing the margin is good according to intuition
2. Implies that only support vectors are important; other training examples are ignorable.
3. Empirically it works very very well.

Support Vectors
are those data points that the margin pushes up against



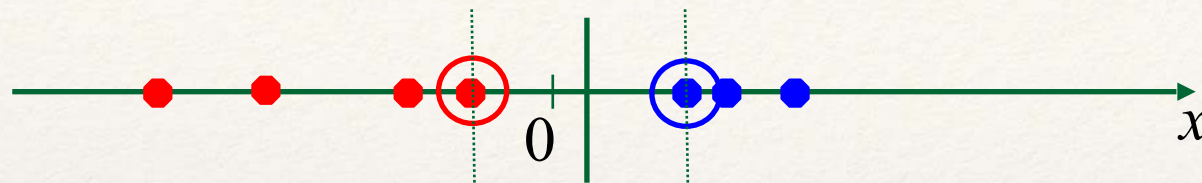
linear classifier
with the, um,
maximum margin.

This is the simplest
kind of SVM
(Called an LSVM)

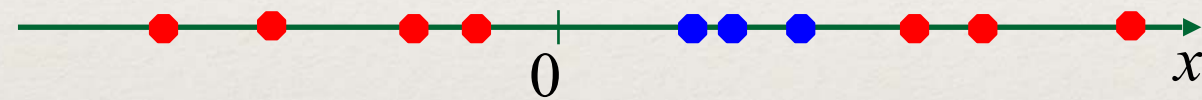
Linear SVM

Non-linear SVMs

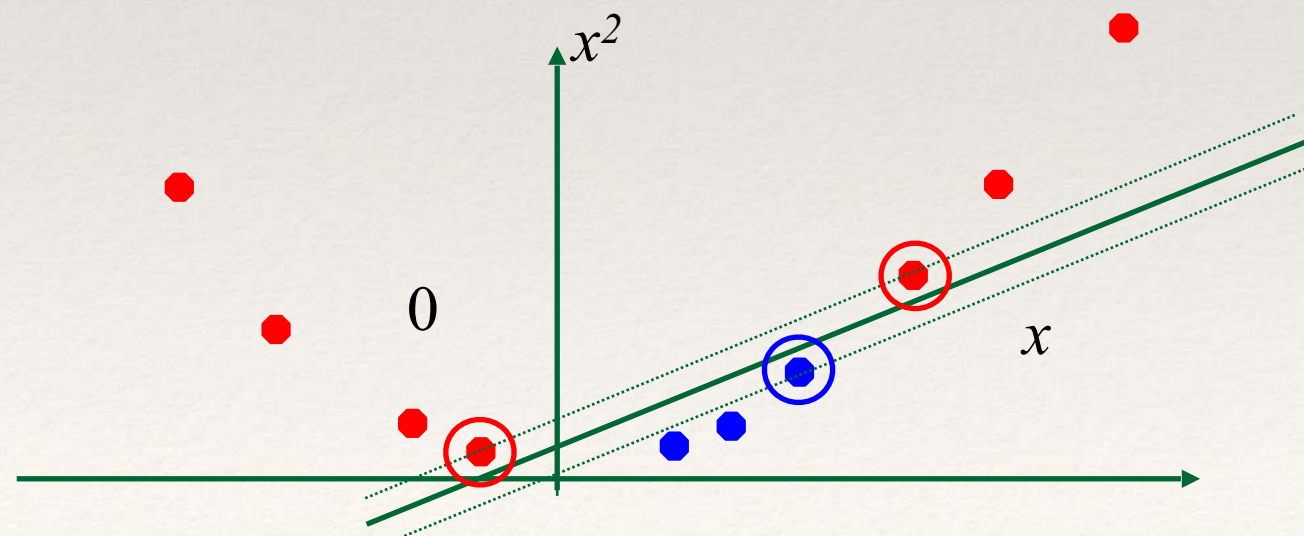
- Datasets that are linearly separable with some noise work out great:



- But what are we going to do if the dataset is just too hard?

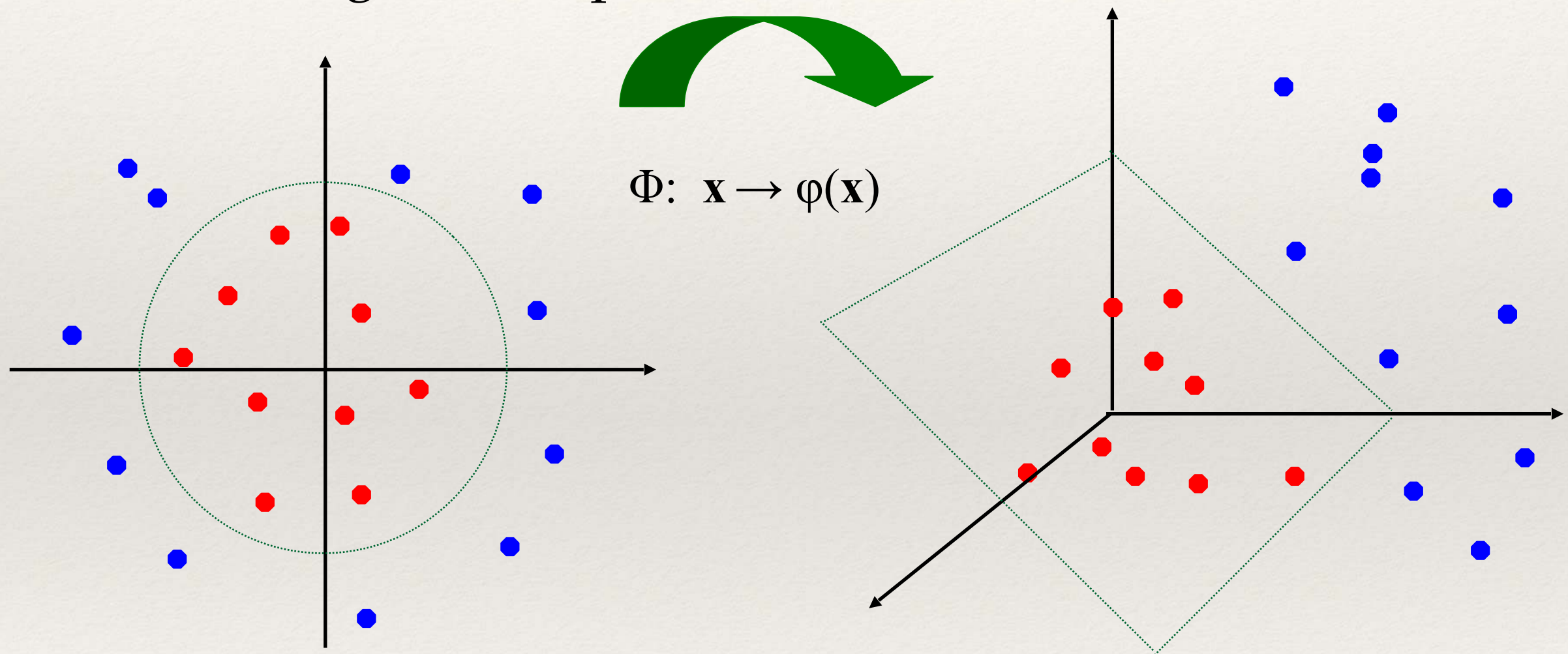


- How about... mapping data to a higher-dimensional space:



Non-linear SVMs: Feature spaces

- General idea: the original input space can always be mapped to some higher-dimensional feature space where the training set is separable:



The “Kernel Trick”

- The linear classifier relies on dot product between vectors $k(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- If every data point is mapped into high-dimensional space via some transformation $\Phi: \mathbf{x} \rightarrow \phi(\mathbf{x})$, the dot product becomes:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

- A *kernel function* is some function that corresponds to an inner product in some expanded feature space.
- Example:

2-dimensional vectors $\mathbf{x} = [x_1 \ x_2]$; let $k(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^2$,

Need to show that $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$:

$$k(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^2,$$

$$= 1 + x_{i1}^2 x_{j1}^2 + 2 x_{i1} x_{j1} x_{i2} x_{j2} + x_{i2}^2 x_{j2}^2 + 2 x_{i1} x_{j1} + 2 x_{i2} x_{j2}$$

$$= [1 \ x_{i1}^2 \ \sqrt{2} x_{i1} x_{i2} \ x_{i2}^2 \ \sqrt{2} x_{i1} \ \sqrt{2} x_{i2}]^T [1 \ x_{j1}^2 \ \sqrt{2} x_{j1} x_{j2} \ x_{j2}^2 \ \sqrt{2} x_{j1} \ \sqrt{2} x_{j2}]$$

$$= \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j), \quad \text{where } \phi(\mathbf{x}) = [1 \ x_1^2 \ \sqrt{2} x_1 x_2 \ x_2^2 \ \sqrt{2} x_1 \ \sqrt{2} x_2]$$

What Functions are Kernels?

- For many functions $k(\mathbf{x}_i, \mathbf{x}_j)$ checking that

$k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$ can be cumbersome.

- Mercer's theorem: Every semi-positive definite symmetric function is a kernel
- Semi-positive definite symmetric functions correspond to a semi-positive definite symmetric Gram matrix:

$\mathbf{K}$ =

$k(\mathbf{x}_1, \mathbf{x}_1)$	$k(\mathbf{x}_1, \mathbf{x}_2)$	$k(\mathbf{x}_1, \mathbf{x}_3)$	...	$k(\mathbf{x}_1, \mathbf{x}_N)$
$k(\mathbf{x}_2, \mathbf{x}_1)$	$k(\mathbf{x}_2, \mathbf{x}_2)$	$k(\mathbf{x}_2, \mathbf{x}_3)$		$k(\mathbf{x}_2, \mathbf{x}_N)$
...	...	...	...	...
$k(\mathbf{x}_N, \mathbf{x}_1)$	$k(\mathbf{x}_N, \mathbf{x}_2)$	$k(\mathbf{x}_N, \mathbf{x}_3)$	...	$k(\mathbf{x}_N, \mathbf{x}_N)$

Examples of Kernel Functions

- Linear: $k(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- Polynomial of power p : $k(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^p$
- Gaussian (radial-basis function network):

$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp \frac{-\|\mathbf{x}_i - \mathbf{x}_j\|^2}{\sigma^2}$$

- Sigmoid: $k(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\beta_0 \mathbf{x}_i^T \mathbf{x}_j + \beta_1)$

SVM Formulation

- ❖ Goal - 1) Correctly classify all training data

$$\left. \begin{aligned} \mathbf{w}^T \phi(\mathbf{x}_n) + b &\geq 1 & \text{if } t_n = +1 \\ \mathbf{w}^T \phi(\mathbf{x}_n) + b &\leq -1 & \text{if } t_n = -1 \end{aligned} \right\} \rightarrow$$
$$t_n(\mathbf{w}^T \phi(\mathbf{x}_n) + b) \geq 1$$

- 2) Define the Margin

$$\frac{1}{\|\mathbf{w}\|} \min_n [t_n(\mathbf{w}^T \phi(\mathbf{x}_n) + b)]$$

- 3) Maximize the Margin

$$\operatorname{argmax}_{\mathbf{w}, b} \left\{ \frac{1}{\|\mathbf{w}\|} \min_n [t_n(\mathbf{w}^T \phi(\mathbf{x}_n) + b)] \right\}$$

- ❖ Equivalently written as

$$\operatorname{argmin}_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{such that } t_n(\mathbf{w}^T \phi(\mathbf{x}_n) + b) \geq 1$$

Solving the Optimization Problem

- Need to optimize a *quadratic* function subject to *linear* constraints.
- Quadratic optimization problems are a well-known class of mathematical programming problems, and many (rather intricate) algorithms exist for solving them.
- The solution involves constructing a *dual problem* where a *Lagrange multiplier* a_n is associated with every constraint in the primary problem:
- The dual problem in this case is maximized

Find $\{a_1, \dots, a_N\}$ such that

$$\tilde{L}(\mathbf{a}) = \sum_{n=1}^N a_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N t_n t_m a_n a_m k(\mathbf{x}_n, \mathbf{x}_m) \text{ maximized}$$

and $\sum_n a_n t_n = 0, \quad a_n \geq 0$

Solving the Optimization Problem

- The solution has the form:

$$\mathbf{w} = \sum_{n=1}^N a_n \phi(\mathbf{x}_n)$$

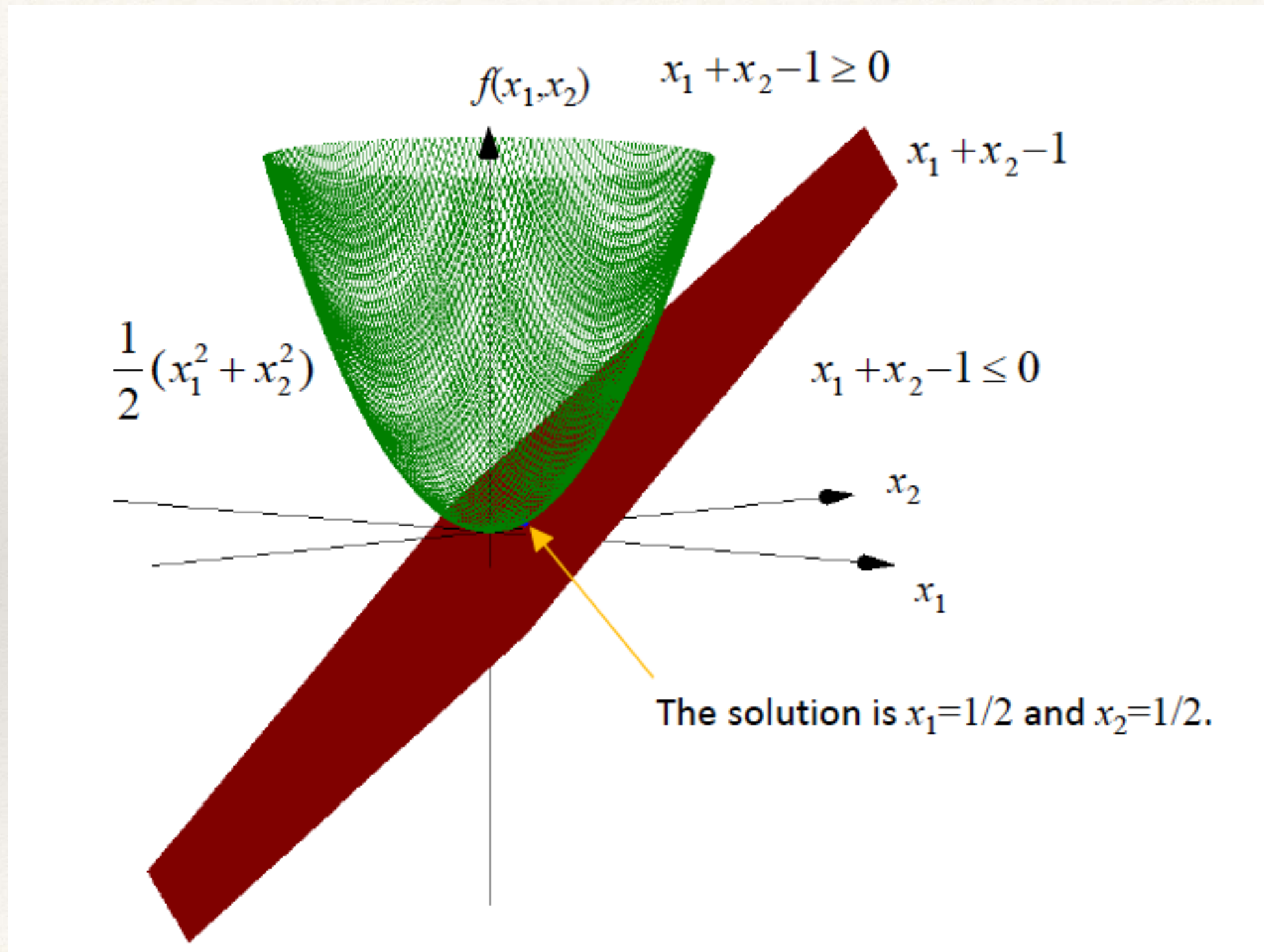
- Each non-zero a_n indicates that corresponding $\mathbf{x}_n$ is a support vector. Let S denote the set of support vectors.

$$b = y(\mathbf{x}_n) - \sum_{m \in S} a_m k(\mathbf{x}_m, \mathbf{x}_n)$$

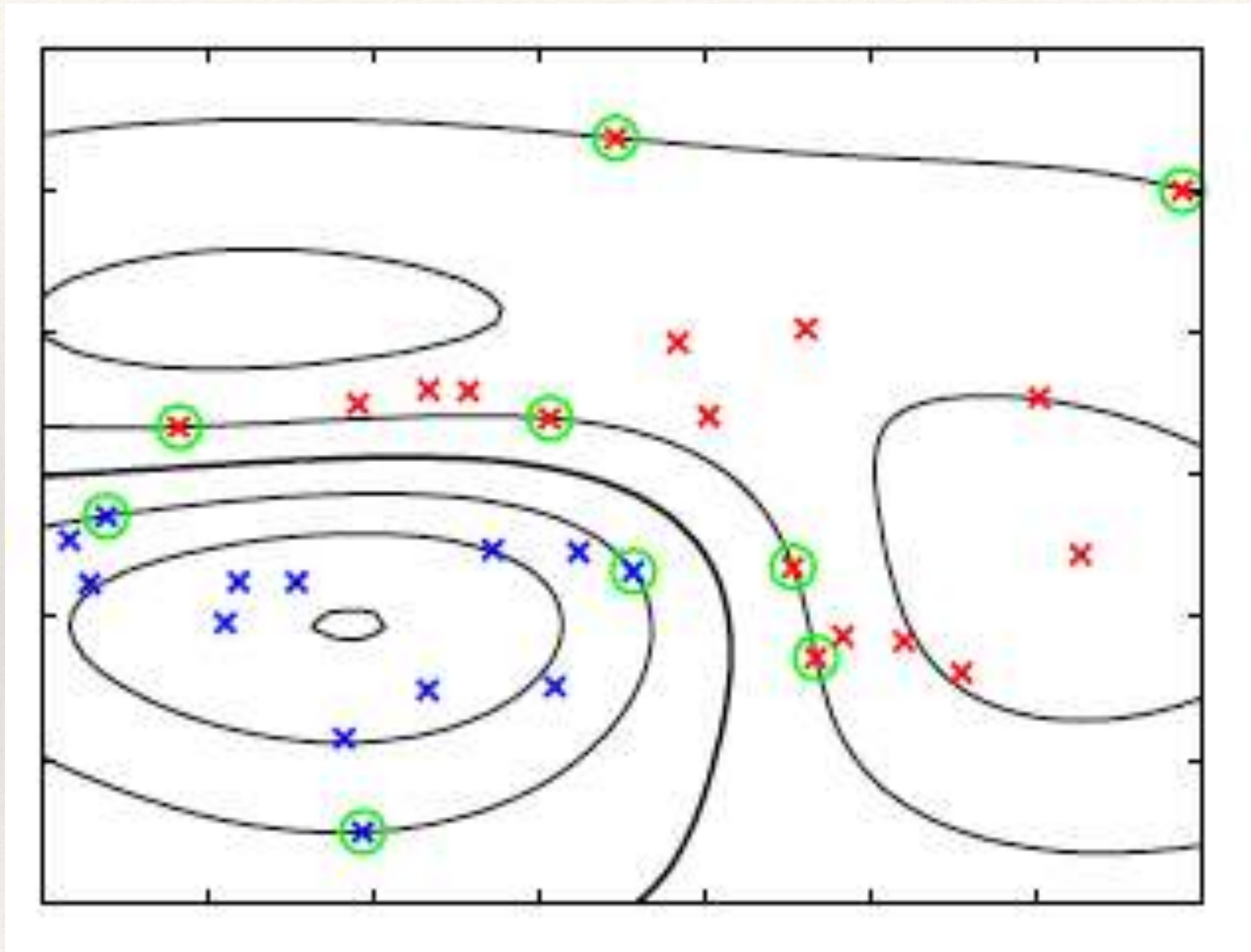
- And the classifying function will have the form:

$$y(\mathbf{x}) = \sum_{n \in S} a_n k(\mathbf{x}_n, \mathbf{x}) + b$$

Solving the Optimization Problem



Visualizing Gaussian Kernel SVM



Overlapping class boundaries

- The classes are not linearly separable - Introducing slack variables ζ_n
- Slack variables are non-negative $\zeta_n \geq 0$
- They are defined using

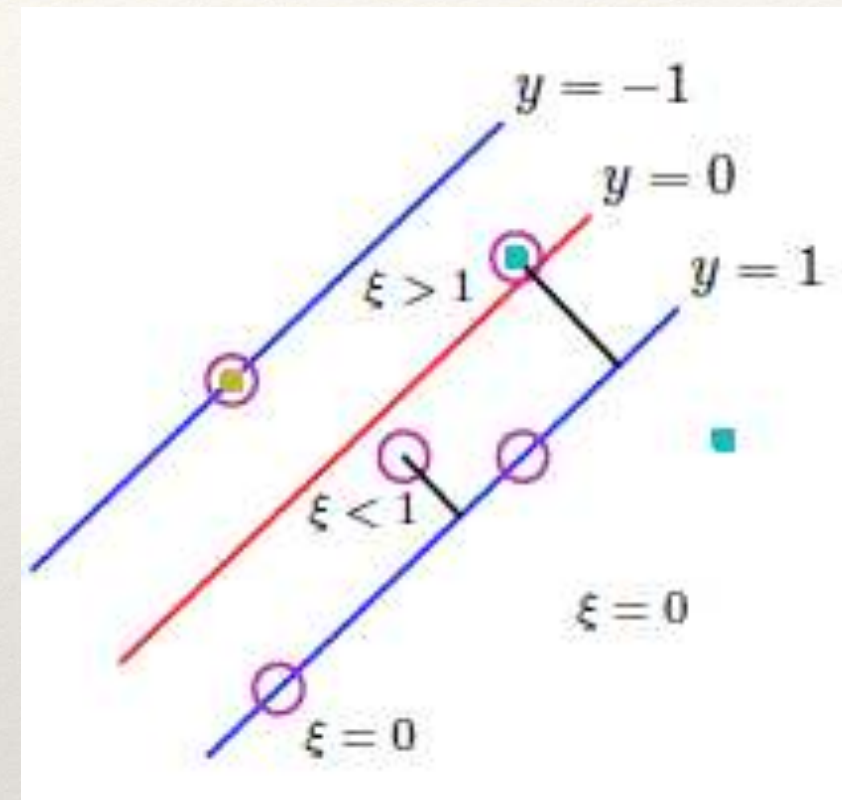
$$t_n y(\mathbf{x}_n) \geq 1 - \zeta_n$$

- The upper bound on mis-classification

$$\sum_n \zeta_n$$

- The cost function to be optimized in this case

$$C \sum_n \zeta_n + \frac{1}{2} \mathbf{w}^T \mathbf{w}$$



SVM Formulation - overlapping classes

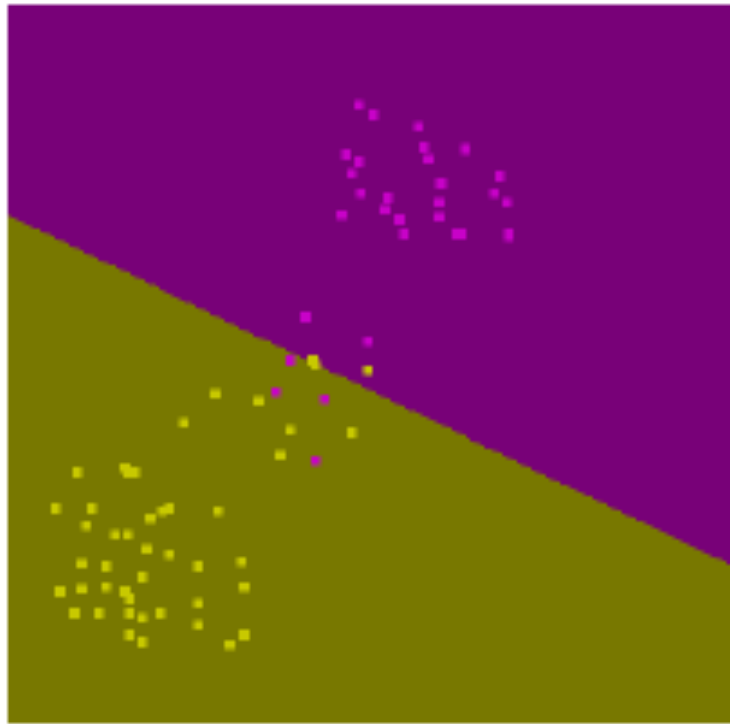
- Formulation very similar to previous case except for additional constraints

$$0 \leq a_n \leq C$$

- Solved using the dual formulation - sequential minimal optimization algorithm
- Final classifier is based on the sign of

$$y(\mathbf{x}) = \sum_{n \in S} a_n k(\mathbf{x}_n, \mathbf{x}) + b$$

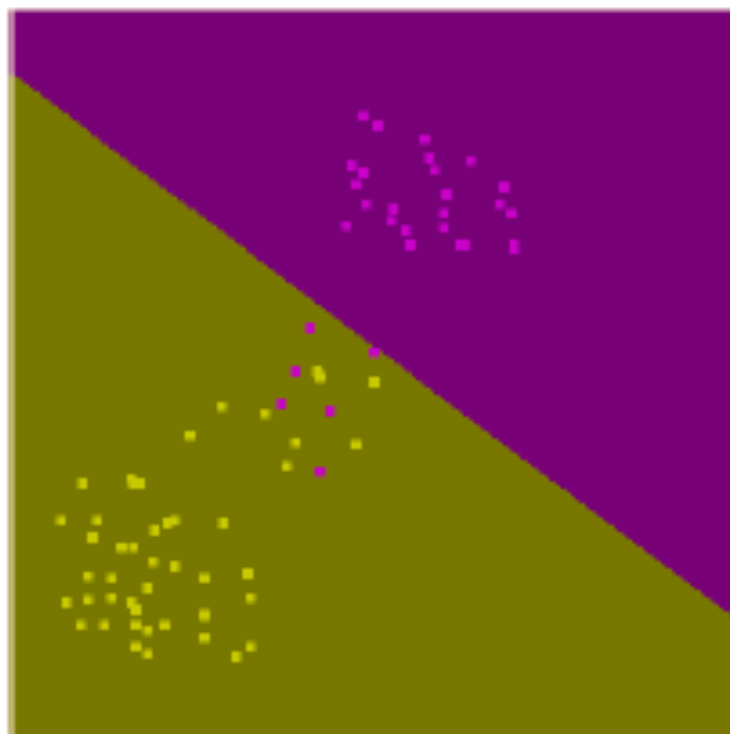
Overlapping class boundaries



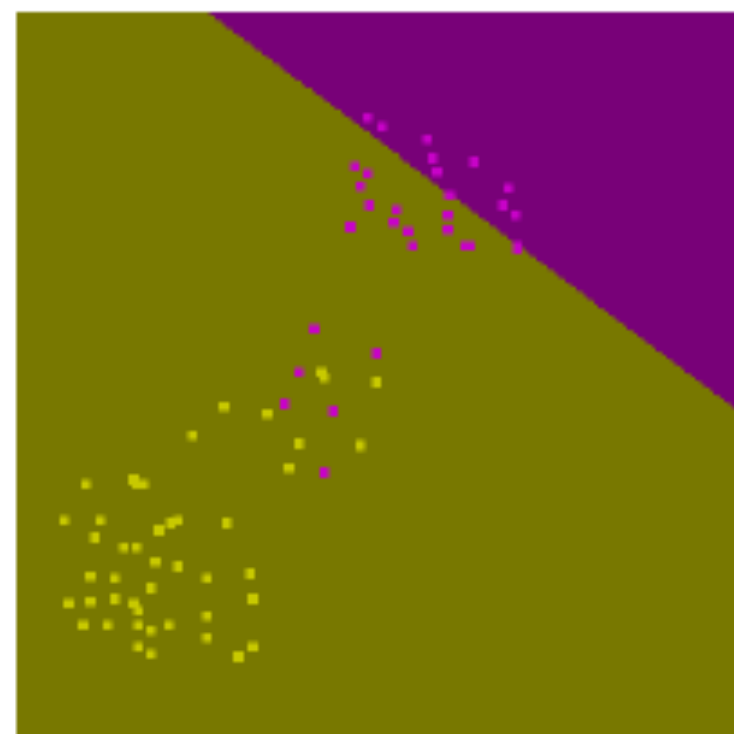
$C=100$



$C=1$



$C=0.15$



$C=0.1$

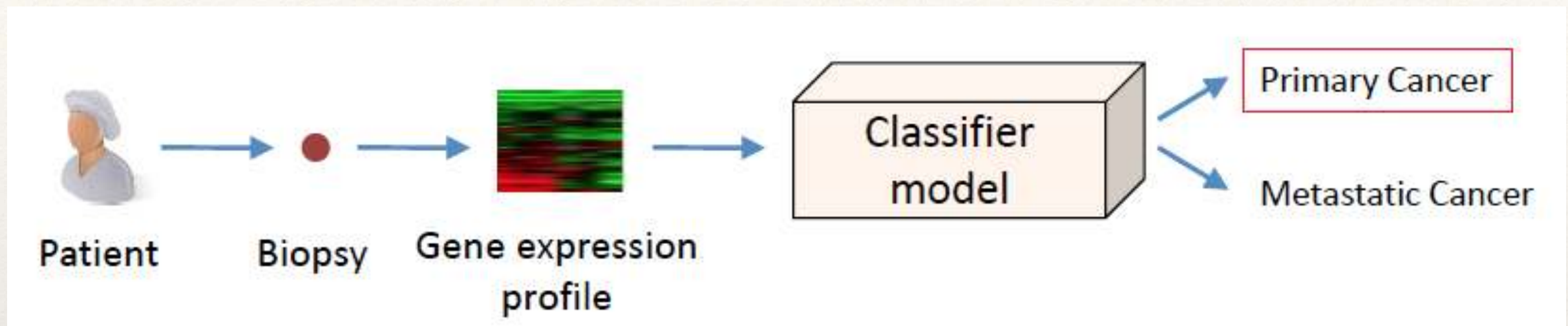
Properties of SVM

- Flexibility in choosing a similarity function
- **Sparseness** of solution when dealing with large data sets
 - only support vectors are used to specify the separating hyperplane
- Ability to **handle large feature spaces**
 - complexity does not depend on the dimensionality of the feature space
- Overfitting can be controlled by soft margin approach
- **Nice math property**: a simple convex optimization problem which is guaranteed to converge to a single global solution
- Feature Selection

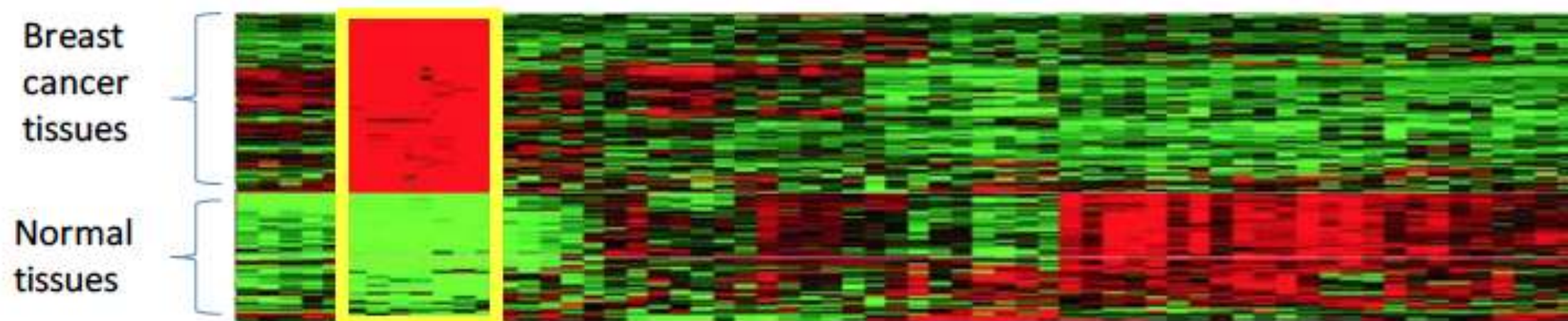
SVM Applications

- SVM has been used successfully in many real-world problems
 - text (and hypertext) categorization
 - image classification
 - bioinformatics (Protein classification, Cancer classification)
 - hand-written character recognition

Application 1: Cancer Classification

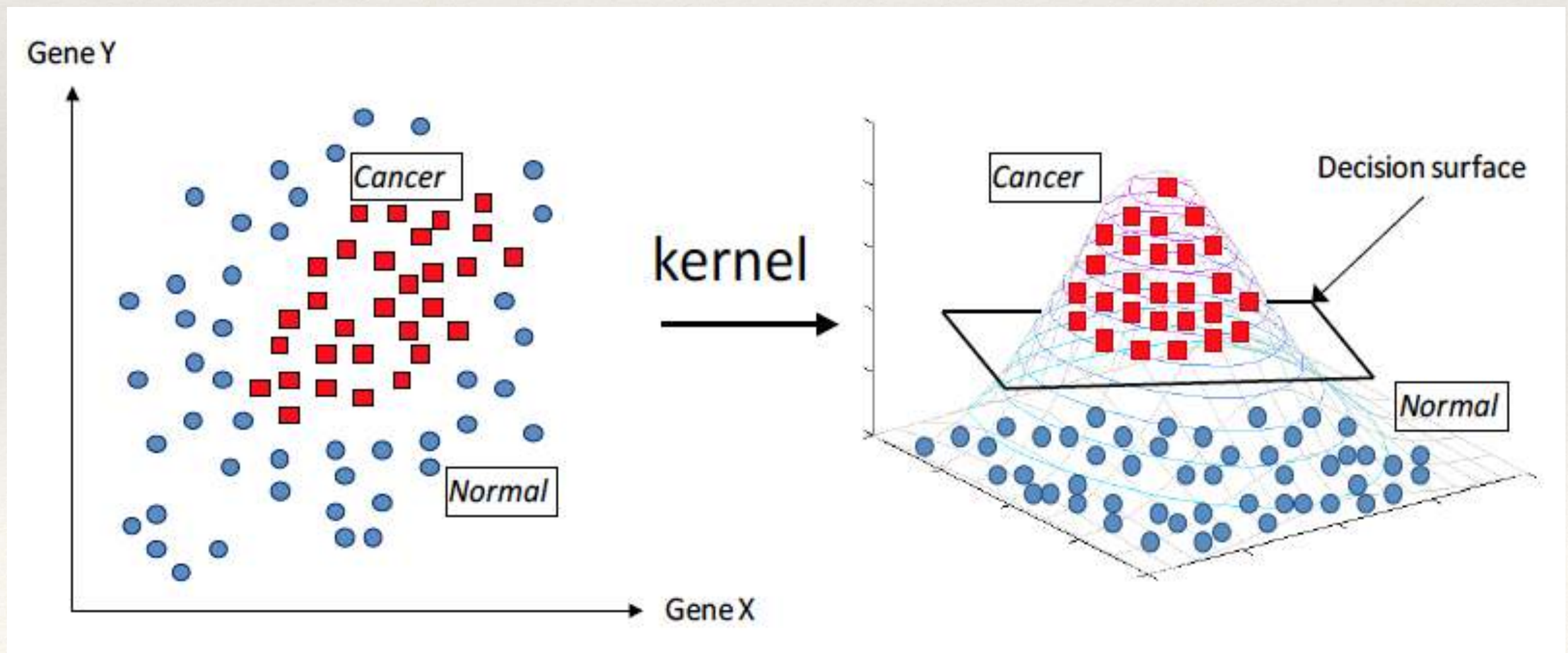
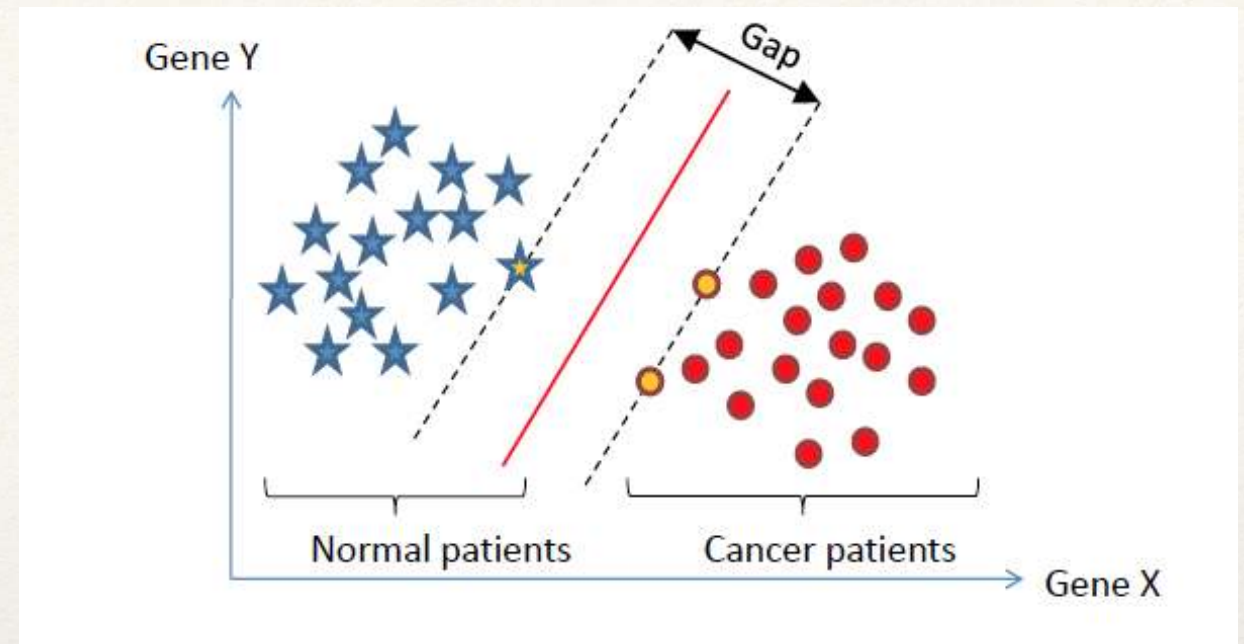


- E.g., find the most compact panel of breast cancer biomarkers from microarray gene expression data for 20,000 genes:

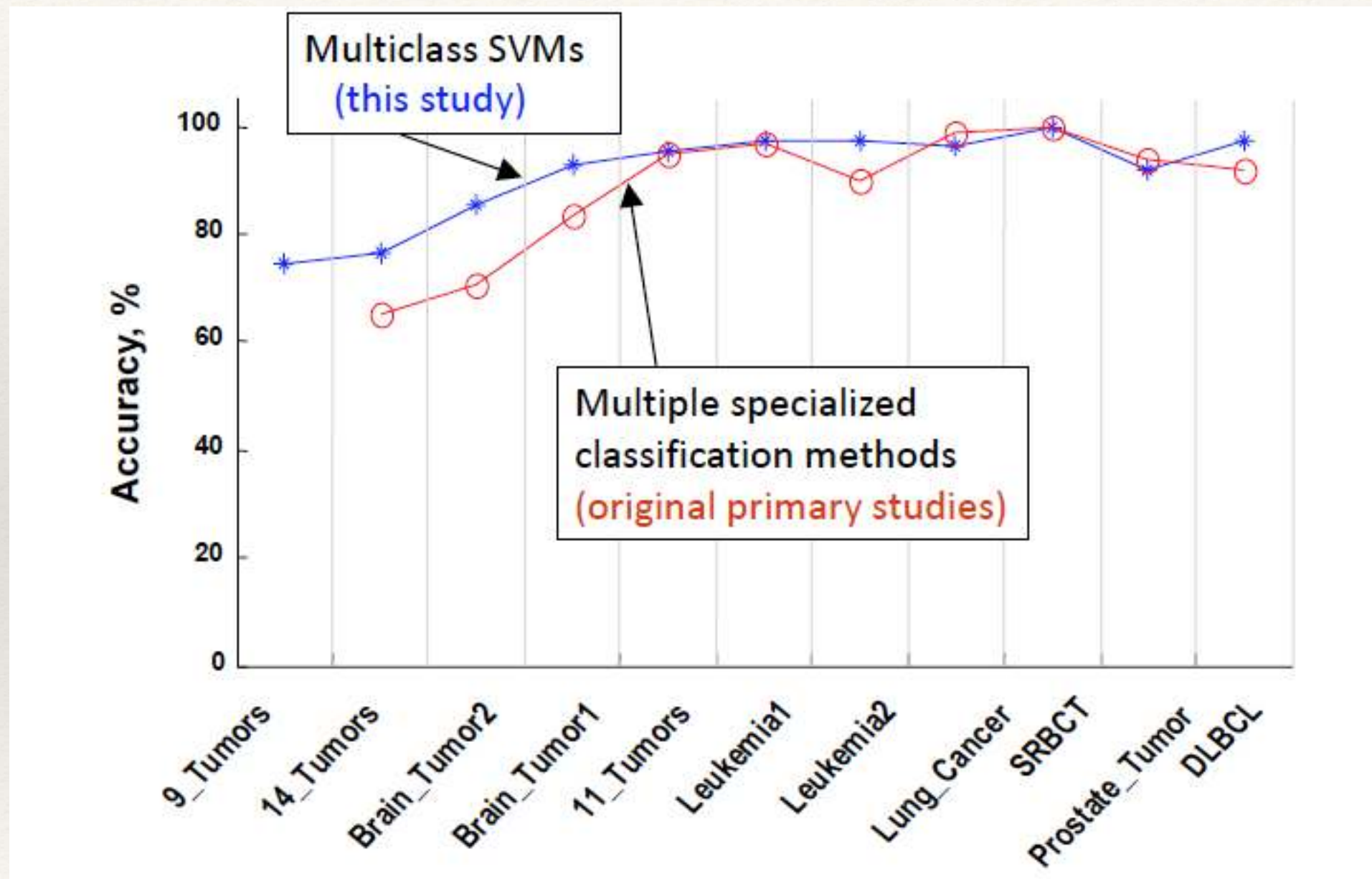


Application 1: Cancer Classification

Linear Versus Non-linear SVMs



Application 1: Cancer Classification



Weakness of SVM

- **It is sensitive to noise**
 - A relatively small number of mislabeled examples can dramatically decrease the performance
- **It only considers two classes**
 - how to do multi-class classification with SVM?
 - Answer:
 - 1) with output m , learn m SVM's
 - SVM 1 learns "Output==1" vs "Output != 1"
 - SVM 2 learns "Output==2" vs "Output != 2"
 - :
 - SVM m learns "Output== m " vs "Output != m "
 - 2) To predict the output for a new input, just predict with each SVM and find out which one puts the prediction the furthest into the positive region.

Application 2: Text Categorization

- Task: The classification of natural text (or hypertext) documents into a fixed number of predefined categories based on their content.
 - email filtering, web searching, sorting documents by topic, etc..
- A document can be assigned to more than one category, so this can be viewed as a series of binary classification problems, one for each category.

Application 2: Text Categorization

IR's vector space model (aka bag-of-words representation)

- A doc is represented by a vector indexed by a pre-fixed set or dictionary of terms
- Values of an entry can be binary or weights

$$\phi_i(x) = \frac{\text{tf}_i \log(\text{idf}_i)}{\kappa},$$

- Doc $x \Rightarrow \phi(x)$

Application 2: Text Categorization

- The distance between two documents is $\langle \phi(x) \phi(z) \rangle$
- $K(x,z) = \langle \phi(x) \phi(z) \rangle$ is a valid kernel, SVM can be used with $K(x,z)$ for discrimination.
- Why SVM?
 - High dimensional input space
 - Few irrelevant features (dense concept)
 - Sparse document vectors (sparse instances)
 - Text categorization problems are linearly separable

Application 2: Text Categorization

					SVM (poly) degree $d =$					SVM (rbf) width $\gamma =$			
	Bayes	Rocchio	C4.5	k-NN	1	2	3	4	5	0.6	0.8	1.0	1.2
earn	95.9	96.1	96.1	97.3	98.2	98.4	98.5	98.4	98.3	98.5	98.5	98.4	98.3
acq	91.5	92.1	85.3	92.0	92.6	94.6	95.2	95.2	95.3	95.0	95.3	95.3	95.4
money-fx	62.9	67.6	69.4	78.2	66.9	72.5	75.4	74.9	76.2	74.0	75.4	76.3	75.9
grain	72.5	79.5	89.1	82.2	91.3	93.1	92.4	91.3	89.9	93.1	91.9	91.9	90.6
crude	81.0	81.5	75.5	85.7	86.0	87.3	88.6	88.9	87.8	88.9	89.0	88.9	88.2
trade	50.0	77.4	59.2	77.4	69.2	75.5	76.6	77.3	77.1	76.9	78.0	77.8	76.8
interest	58.0	72.5	49.1	74.0	69.8	63.3	67.9	73.1	76.2	74.4	75.0	76.2	76.1
ship	78.7	83.1	80.9	79.2	82.0	85.4	86.0	86.5	86.0	85.4	86.5	87.6	87.1
wheat	60.6	79.4	85.5	76.6	83.1	84.5	85.2	85.9	83.8	85.2	85.9	85.9	85.9
corn	47.3	62.2	87.7	77.9	86.0	86.5	85.3	85.7	83.9	85.1	85.7	85.7	84.5
microavg.	72.0	79.9	79.4	82.3	84.2	85.1	85.9	86.2	85.9	86.4	86.5	86.3	86.2
					combined: 86.0					combined: 86.4			

Application 3: Handwriting Recognition

For example MNIST hand-writing recognition.

60,000 training examples, 10000 test examples, 28x28.

Linear SVM has around 8.5% test error.

Polynomial SVM has around 1% test error.



SVMs : full MNIST results

Classifier	Test Error
linear	8.4%
3-nearest-neighbor	2.4%
RBF-SVM	1.4 %

Some Considerations

- Choice of kernel
 - Gaussian or polynomial kernel is default
 - if ineffective, more elaborate kernels are needed
 - domain experts can give assistance in formulating appropriate similarity measures
- Choice of kernel parameters
 - e.g. σ in Gaussian kernel
 - σ is the distance between closest points with different classifications
 - In the absence of reliable criteria, applications rely on the use of a validation set or cross-validation to set such parameters.
- Optimization criterion – Hard margin v.s. Soft margin
 - a lengthy series of experiments in which various parameters are tested

Software

30 SVMs : software

Lots of SVM software:

- LibSVM (C++)
- SVMLight (C)

As well as complete machine learning toolboxes that include SVMs:

- Torch (C++)
- Spider (Matlab)
- Weka (Java)

All available through www.kernel-machines.org.

Miscellaneous Announcements

- Firm deadline on 28-10-2016 (noon)
- No class on 26-10-2016 (we will compensate for one class some time later).
- Implementation of likelihoods
 - Use the logarithm of likelihoods and apply sum (otherwise will go out machine precision)
- HMM
 - use implementation issues and scaling procedure discussed in Sec. 6.12.2 (Rabiner and Juang Book)
 - use flat start to begin the HMM iteration.